

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I



BÁO CÁO BÀI TẬP LỚN
BỘ MÔN CÁC HỆ THỐNG PHÂN TÁN
ĐỀ TÀI:
PHÁT TRIỂN HỆ THỐNG CHAT NGANG HÀNG P2P

Giảng viên hướng dẫn: Thầy Kim Ngọc Bách
Nhóm sinh viên: Bùi Quang Huy - B22DCCN376
Thái Hữu Khởi – B22DCCN472
Đinh Bá Đạt – B21DCCN210

MỤC LỤC

MỤC LỤC	2
CHƯƠNG 1: KIẾN TRÚC HỆ THỐNG	1
1.1: Tổng quan kiến trúc	1
1.2: Thành phần Tracker Server	2
1.3: Thành phần Peer Node	3
1.4: Giao diện người dùng	3
CHƯƠNG 2: GIAO THỨC TRAO ĐỔI THÔNG ĐIỆP	4
2.1: Định dạng thông điệp	4
2.2: Danh sách loại thông điệp	4
2.2.1. Thông điệp Tracker - Peer	4
2.2.2. Thông điệp Peer - Peer	5
2.3. Luồng giao tiếp chính	6
2.3.1. Luồng đăng ký và kết nối mạng	6
2.3.2. Luồng chat trực tiếp có xác nhận.....	7
2.3.3. Luồng gửi tin qua peer trung gian	8
2.3.4. Luồng store-and-forward khi peer offline	9
2.4. Mã hóa thông điệp	10
2.5. Truyền file giữa các peer	10
CHƯƠNG 3: CƠ CHẾ PEER DISCOVERY	11
3.1. Đăng ký trực tiếp với Tracker	11
3.2. Đăng ký qua peer trung gian.....	12
3.3. Truy vấn danh sách peer online	12
3.5. Cập nhật danh sách peer real-time.....	14
3.5.1. Thiết kế tổng thể	14
3.5.2. TrackerPushBroadcaster	14
3.5.3. Luồng sự kiện khi peer mới join.....	16
3.5.4. Luồng sự kiện khi peer rời mạng.....	17
3.5.5. Xử lý trên phía Peer nhận	17
CHƯƠNG 4: XỬ LÝ LỖI HỆ THỐNG	18
4.1. Truyền tin đáng tin cậy	18
4.2. Lưu tin khi peer offline.....	19

4.3. Chống tin trùng	19
CHƯƠNG 5: THỰC NGHIỆM VÀ ĐÁNH GIÁ	20
5.1. Kịch bản thử nghiệm	20
5.5.1: Thử nghiệm chức năng cơ bản	20
5.5.1: Thử nghiệm chức năng cơ bản	23
5.2. Mô phỏng churn mạng.....	24
5.3. Đánh giá kết quả	24
Tài liệu tham khảo	27

DANH MỤC BẢNG BIỂU

Bảng 1. Các lớp chính của Tracker Server	2
Bảng 2. Các lớp chính của Peer Node	3
Bảng 3. Cấu trúc lớp Message	4
Bảng 4. Giao thức Peer → Tracker (request-response)	4
Bảng 5. Giao thức Tracker → Peer (push chủ động)	5
Bảng 6. Giao thức Peer ↔ Peer	5
Bảng 7. Thông số mã hóa AES-GCM	10
Bảng 8. Thông số cơ chế Heartbeat	13
Bảng 9 Thông số cơ chế Reliable Delivery	18
Bảng 10. Kịch bản thử nghiệm chức năng cơ bản	20
Bảng 11. Kịch bản thử nghiệm xử lý lỗi	23
Bảng 12. Thông số ChurnSimulator	24

DANH MỤC HÌNH ẢNH

Hình 1. Sơ đồ kiến trúc tổng quan hệ thống P2P Chat	1
Hình 2. Luồng đăng ký và duy trì kết nối.....	6
Hình 3. Luồng chat trực tiếp với cơ chế ACK và fallback offline	7
Hình 4. Luồng relay với cơ chế store-and-forward	8
Hình 5. Luồng store-and-forward khi peer offline	9
Hình 6. Luồng đăng ký trực tiếp với Tracker	11
Hình 7 Luồng đăng ký qua peer trung gian.....	12
Hình 8. Cơ chế Heartbeat và phát hiện peer mất kết nối.....	14
Hình 9 Luồng push PEER_JOINED khi peer mới tham gia mạng	16
Hình 10. Luồng push PEER_LEFT khi peer rời mạng	17
Hình 11 Cơ chế retry và xác nhận ACK.....	18
Hình 12. TC-01. Đăng ký trực tiếp với Tracker	21
Hình 13. TC-02. Đăng ký qua peer relay	21
Hình 14. TC-03. Chat một-một	21
Hình 15. TC-04. Chat Nhóm	22
Hình 16. TC-09. Gửi tin khi peer đích offline.....	22

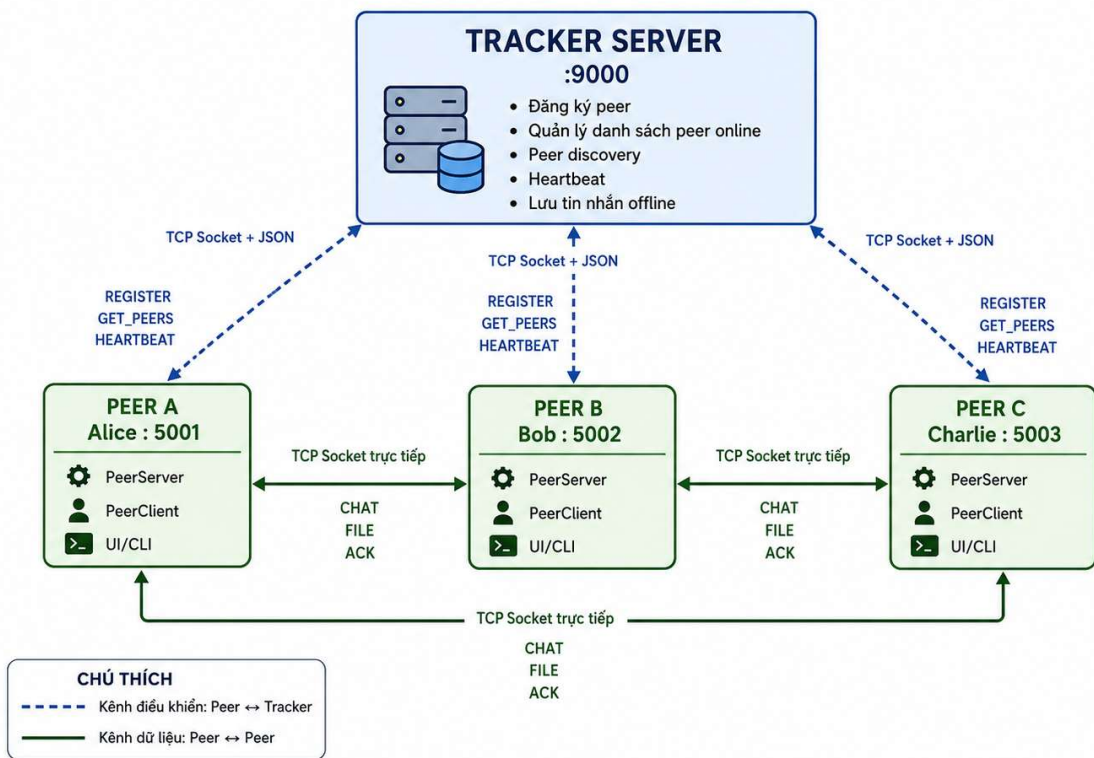
CHƯƠNG 1: KIẾN TRÚC HỆ THỐNG

1.1: Tổng quan kiến trúc

Hệ thống được xây dựng theo mô hình Hybrid P2P: kết hợp một Tracker Server tập trung để hỗ trợ khám phá peer và lưu tin nhắn offline, cùng với giao tiếp ngang hàng trực tiếp giữa các peer sau khi đã biết địa chỉ của nhau.

Toàn bộ giao tiếp sử dụng TCP socket để đảm bảo tin cậy ở tầng vận chuyển. Dữ liệu được tuần tự hóa sang JSON bằng thư viện Gson và truyền dưới dạng một dòng văn bản trên mỗi kết nối TCP.

Sơ đồ kiến trúc hệ thống P2P Chat



Tracker hỗ trợ khám phá peer và quản lý trạng thái; dữ liệu chat/file được truyền trực tiếp giữa các peer qua TCP socket.

Hình 1. Sơ đồ kiến trúc tổng quan hệ thống P2P Chat

1.2: Thành phần Tracker Server

Tracker Server đóng vai trò bootstrap node — điểm vào duy nhất mà một peer mới cần biết để tham gia mạng. Nó không tham gia vào việc chuyển tiếp tin nhắn giữa các peer mà chỉ cung cấp dịch vụ thư mục (directory service) và lưu trữ tin nhắn offline.

Bảng 1. Các lớp chính của Tracker Server

Lớp	Vai trò
TrackerServer	Lắng nghe cổng 9000; phân phối mỗi kết nối đến vào thread pool (10 threads); khởi tạo TrackerPushBroadcaster (CachedThreadPool nội bộ) và khởi động TrackerStalePeerMonitor làm daemon thread.
TrackerState	Lưu trữ toàn bộ trạng thái: danh sách peer online, thời gian heartbeat cuối, hàng đợi tin offline. Mọi trường dùng ConcurrentHashMap để an toàn đa luồng.
TrackerClientConnection	Xử lý một kết nối TCP: đọc một dòng JSON, giải mã thành Message, gọi TrackerMessageDispatcher.
TrackerMessageDispatcher	Điều phối message tới handler tương ứng theo trường type (Strategy Pattern).
TrackerHandlerContext	Đóng gói context của mỗi handler: TrackerState, Gson, PrintWriter, TrackerPushBroadcaster.
TrackerPushBroadcaster	Gửi push notification (PEER_JOINED / PEER_LEFT) tới tất cả peer đang online qua TCP ngắn (fire-and-forget, timeout 2 giây).
TrackerStalePeerMonitor	Daemon thread kiểm tra mỗi 10 giây; xóa peer nếu lastSeen > 15 giây; gọi broadcaster.broadcast(PEER_LEFT).

Cấu trúc dữ liệu của TrackerState:

ConcurrentHashMap<String, PeerInfo> online	onlinePeers	// peerId → peer đang online
ConcurrentHashMap<String, PeerInfo> từng đăng ký (kể cả offline)	knownPeers	// peerId → tất cả peer
ConcurrentHashMap<String, Long> heartbeat cuối	lastSeenPeers	// peerId → thời điểm
ConcurrentHashMap<String, List<Message>>	offlineMessages	// peerId → danh sách tin nhắn chờ

Việc dùng ConcurrentHashMap đảm bảo an toàn khi nhiều thread đọc/ghi đồng thời mà không cần khóa toàn bộ cấu trúc, giúp Tracker xử lý nhiều kết nối song song hiệu quả.

1.3: Thành phần Peer Node

Mỗi peer vừa đóng vai trò nhận tin đến, vừa đóng vai trò gửi tin đi, thực hiện đồng thời trên các thread khác nhau.

Bảng 2. Các lớp chính của Peer Node

Lớp	Vai trò
PeerServer	Thread lắng nghe tại cổng do người dùng chọn; xử lý mỗi kết nối đến trong thread pool (20 threads); khử trùng lặp bằng DeliveryIdempotencyTracker.
PeerClient	Gửi message đến một địa chỉ TCP cụ thể; hỗ trợ retry (3 lần, timeout 3 giây/lần, delay 400ms) và xác nhận ACK qua ReliableDeliveryHelper.
PeerMessagingAdapter	Lớp trung gian bọc PeerClient, cung cấp sendTracker(), sendReliable(), sendPeer() với đầy đủ thông số cấu hình.
PeerMessageDispatcher	Điều phối message inbound tới handler (ChatInboundHandler, FileChunkInboundHandler, PeerChangedInboundHandler, RelayInboundHandler, RelayRegisterInboundHandler).
DeliveryIdempotencyTracker	LRU Map (4096 mục) lưu khóa "type senderId messageId"; bỏ qua message đã xử lý nhưng vẫn gửi ACK.
PeerApplicationServices	Facade tập hợp tất cả service: DirectChatService, GroupChatService, FileSendService, RelaySendService, OfflineMessageService, PeerDirectoryService.

1.4: Giao diện người dùng

Ngoài giao diện dòng lệnh, hệ thống cung cấp thêm giao diện đồ họa JavaFX 21 với thiết kế dark theme. Hai giao diện dùng chung toàn bộ lớp service và handler — chỉ khác nhau ở cách hiển thị kết quả.

Màn hình đăng nhập: Người dùng nhập Peer ID, port lắng nghe, và chọn phương thức tham gia mạng (trực tiếp hoặc qua relay peer).

Màn hình chat chính: Bảng peer online bên trái cho phép click để tự động điền IP/Port; danh sách này cập nhật real-time khi nhận PEER_JOINED / PEER_LEFT từ Tracker.

Các thao tác mạng (gửi tin, kéo danh sách peer, ...) được thực hiện trên background thread riêng biệt để không chặn JavaFX Application Thread. Kết quả được cập nhật lên UI qua Platform.runLater().

CHƯƠNG 2: GIAO THỨC TRAO ĐỔI THÔNG ĐIỆP

2.1: Định dạng thông điệp

Toàn bộ thông điệp trong hệ thống được biểu diễn bằng lớp Message và tuân tự hóa sang JSON bởi thư viện Gson. Mỗi kết nối TCP truyền đúng một thông điệp — một dòng JSON kết thúc bằng ký tự xuống dòng (newline-delimited protocol).

```
{
  "type":      "CHAT",
  "senderId":  "Alice",
  "content":   "U2FsdGVkX1+...", // nội dung (có thể mã hóa AES-Base64)
  "messageId": "550e8400-e29b-41d4-a716-446655440000" // UUID ngẫu nhiên
}
```

Bảng 3. Cấu trúc lớp Message

Trường	Kiểu	Mô tả
type	String	Loại thông điệp, quyết định handler nào xử lý
senderId	String	Peer ID của người gửi
content	String	Nội dung; ý nghĩa phụ thuộc vào type (JSON, Base64-AES, plain text, ...)
messageId	String	UUID ngẫu nhiên dùng cho idempotency và ACK matching

2.2: Danh sách loại thông điệp

2.2.1. Thông điệp Tracker - Peer

Ngoài giao thức request-response thông thường (Peer gửi → Tracker phản hồi), Tracker còn chủ động push PEER_JOINED và PEER_LEFT tới tất cả peer đang online khi có thay đổi topology mạng.

Bảng 4. Giao thức Peer → Tracker (request-response)

Type (gửi đi)	Phản hồi	Mô tả
REGISTER	REGISTER_OK / REGISTER_FAIL	Peer đăng ký với Tracker; content chứa JSON PeerInfo (id, ip, port)
GET_PEERS	PEER_LIST	Yêu cầu danh sách tất cả peer đang online; phản hồi là JSON array PeerInfo[]

GET_KNOWN_PEERS	KNOWN_PEER_LIST	Yêu cầu tất cả peer đã từng đăng ký (kể cả đang offline)
HEARTBEAT	HEARTBEAT_OK	Báo hiệu "tôi vẫn online"; Tracker cập nhật lastSeen timestamp
QUIT	QUIT_OK	Peer đăng xuất chủ động; Tracker xóa peer và broadcast PEER_LEFT
STORE_OFFLINE	ACK / NACK	Lưu tin nhắn mã hóa cho peer offline; content = "targetId encrypted" hoặc "ip port encrypted"
PULL_OFFLINE	OFFLINE_MESSAGES / NO_OFFLINE	Lấy toàn bộ hàng đợi tin offline; Tracker xóa queue sau khi trả về

Bảng 5. Giao thức Tracker → Peer (push chủ động)

Type (Tracker push → Peer)	Phản hồi	Mô tả
PEER_JOINED	—	Tracker push tới tất cả peer khi có peer mới REGISTER thành công; content là JSON PeerInfo của peer mới
PEER_LEFT	—	Tracker push tới tất cả peer còn lại khi một peer gửi QUIT hoặc bị StalePeerMonitor xóa do timeout

2.2.2. Thông điệp Peer - Peer

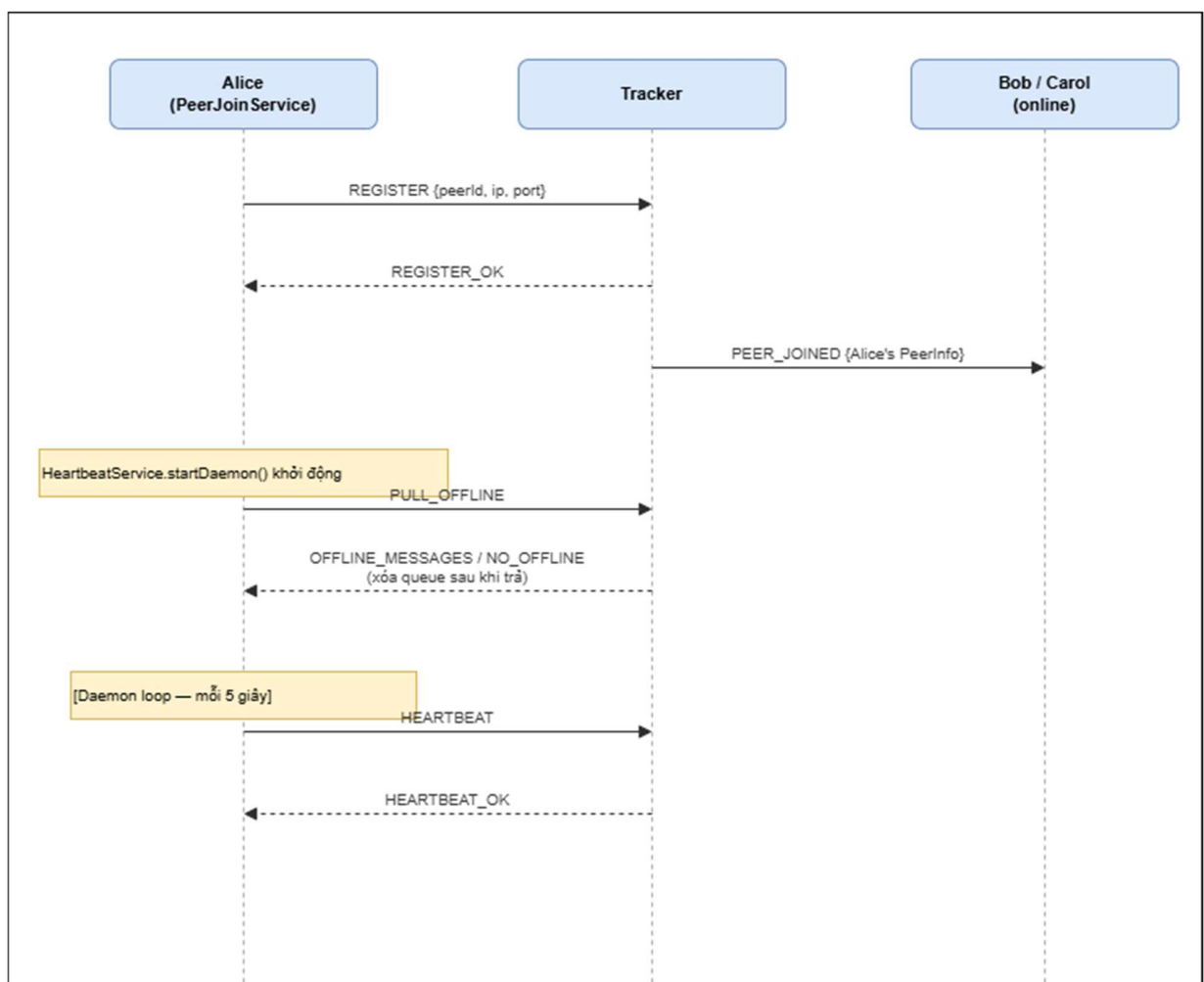
Bảng 6. Giao thức Peer ↔ Peer

Type	Phản hồi	Mô tả
CHAT	ACK	Tin nhắn trực tiếp một-một; content là chuỗi AES/GCM-Base64 của plaintext
GROUP_CHAT	ACK	Tin nhắn nhóm; người gửi gửi đến tất cả peer đích song song bằng CompletableFuture
FILE_CHUNK	ACK	Một chunk 512KB của file; content = "transferId fileName chunkIdx totalChunks Base64Data"
FILE_RESUME_REQ	ACK (nextChunk)	Hỏi receiver chunk tiếp theo cần nhận để resume transfer bị gián đoạn

RELAY	ACK	Yêu cầu peer trung gian chuyển tiếp tin; content = "targetId AES-Base64"
RELAY_REGISTER	REGISTER_OK/FAIL	Yêu cầu peer đăng ký hộ với Tracker; content là JSON PeerInfo của peer mới
ACK	—	Xác nhận đã nhận và xử lý thành công; messageId phải trùng với messageId gốc

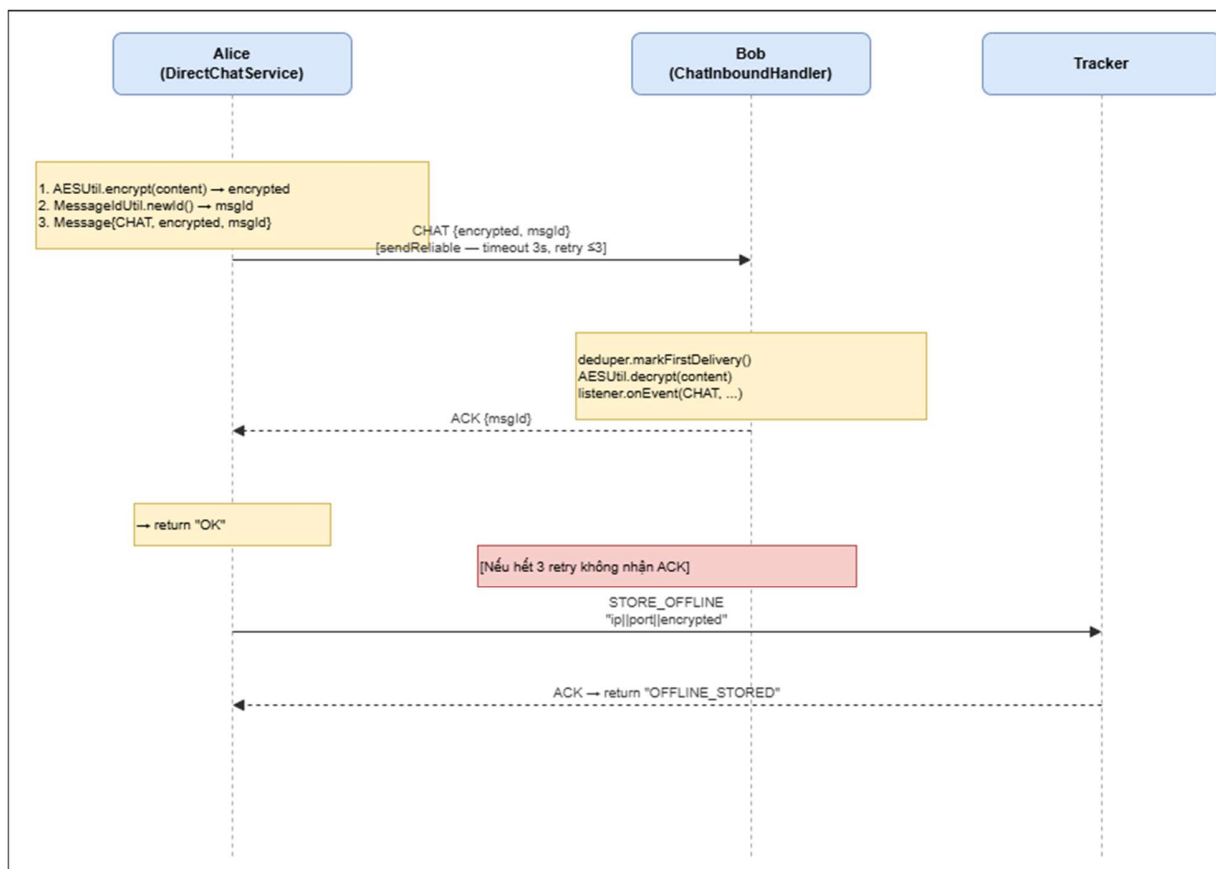
2.3. Luồng giao tiếp chính

2.3.1. Luồng đăng ký và kết nối mạng



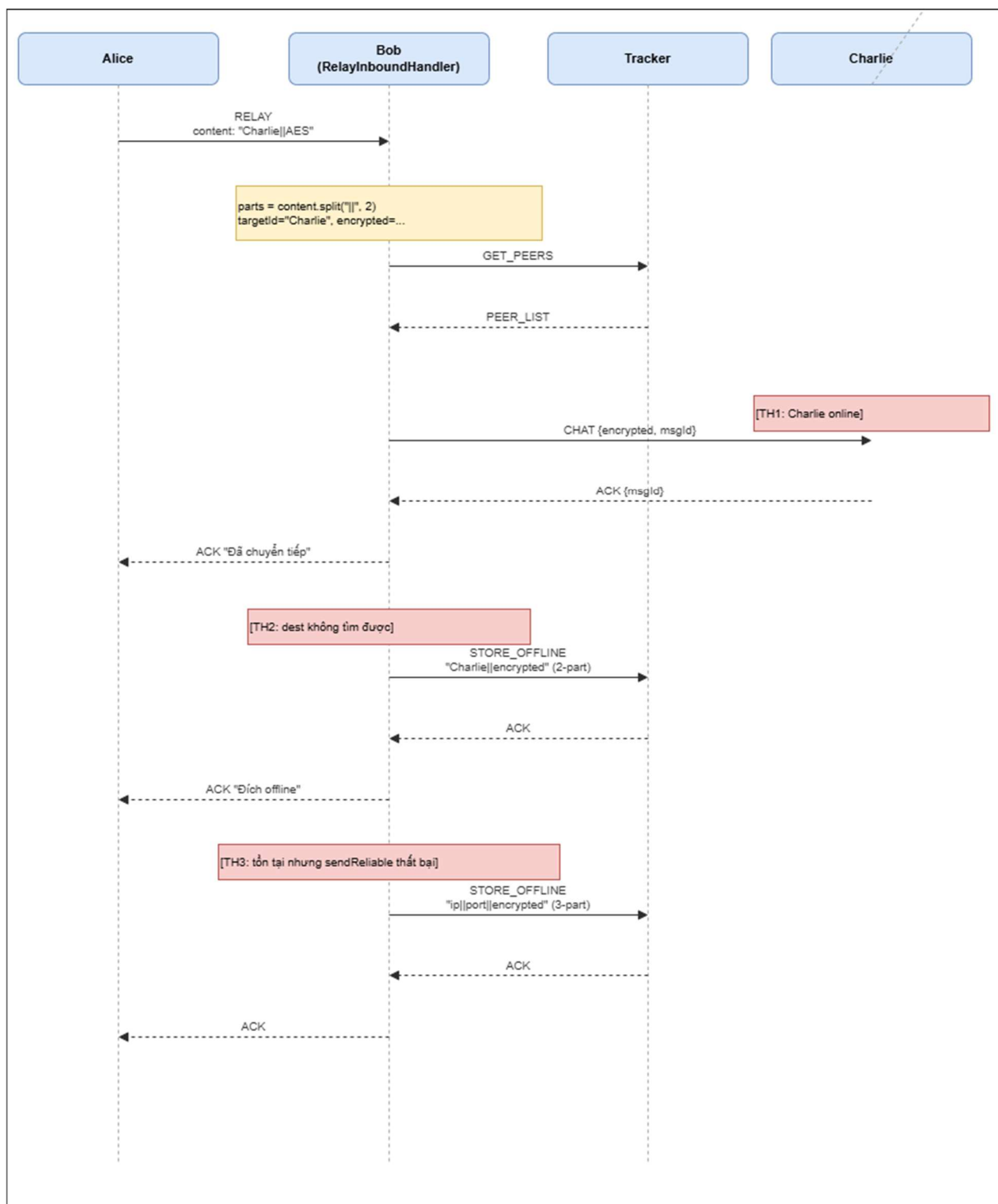
Hình 2. Luồng đăng ký và duy trì kết nối

2.3.2. Luồng chat trực tiếp có xác nhận



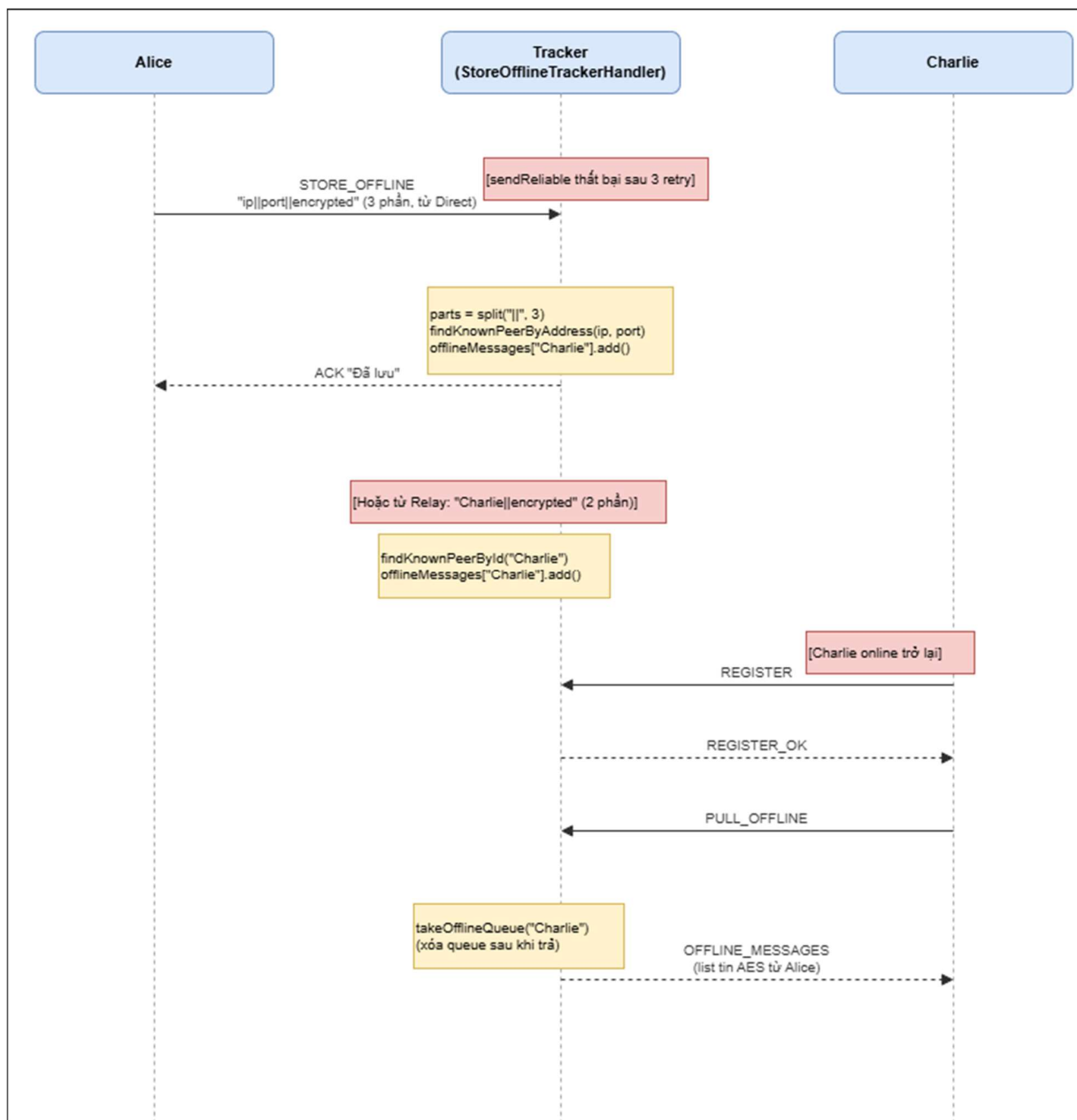
Hình 3. Luồng chat trực tiếp với cơ chế ACK và fallback offline

2.3.3. Luồng gửi tin qua peer trung gian



Hình 4. Luồng relay với cơ chế store-and-forward

2.3.4. Luồng store-and-forward khi peer offline



Hình 5. Luồng store-and-forward khi peer offline

2.4. Mã hóa thông điệp

Mọi nội dung tin nhắn (CHAT, GROUP_CHAT, RELAY) đều được mã hóa trước khi gửi qua mạng bằng lớp AESUtil. Ngay cả khi lưu trên Tracker (STORE_OFFLINE), Tracker chỉ thấy ciphertext.

Bảng 7. Thông số mã hóa AES-GCM

Thuộc tính	Giá trị
Thuật toán	AES/GCM/NoPadding
Độ dài khóa	128-bit (16 byte)
IV (nonce)	12 byte ngẫu nhiên (SecureRandom) mỗi lần mã hóa
GCM tag length	128 bit — đảm bảo tính toàn vẹn (authentication)
Định dạng output	Base64(IV ciphertext GCM-tag)
Khóa hiện tại	"P2P_Chat_Key_123" (hardcoded — nên thay bằng DH key exchange)
Lớp triển khai	AESUtil (encrypt/decrypt với javax.crypto.Cipher)

Chế độ GCM (Galois/Counter Mode) là authenticated encryption — ngoài bảo mật nội dung còn đảm bảo tính toàn vẹn: bất kỳ thay đổi nào trong ciphertext sẽ khiến giải mã thất bại.

2.5. Truyền file giữa các peer

File được truyền trực tiếp qua kết nối TCP P2P theo giao thức chunking với hỗ trợ resume:

- Sender chia file thành các chunk 512KB; mỗi chunk được Base64-encode và đóng gói vào FILE_CHUNK message với transferId (UUID), chỉ số chunk, tổng số chunk.
- Gửi từng chunk qua sendReliable(); nhận ACK trước khi gửi chunk tiếp theo.
- Receiver (FileChunkInboundHandler) lưu từng chunk vào thư mục tạm downloads_<port>/tmp/<transferId>/<idx>.
- Khi nhận đủ tất cả chunk (kiểm tra qua TreeSet<Integer>): ghép theo thứ tự vào file cuối tại downloads_<port>/<fileName>, xóa thư mục tạm.
- Bảo vệ path traversal: fileName được làm sạch bằng Paths.get(name).getFileName() trước khi dùng.

Kích thước file tối đa: 50 MB. Timeout transfer: 24 giờ. FileTransferRegistry tự dọn dẹp transfer hết hạn mỗi giờ.

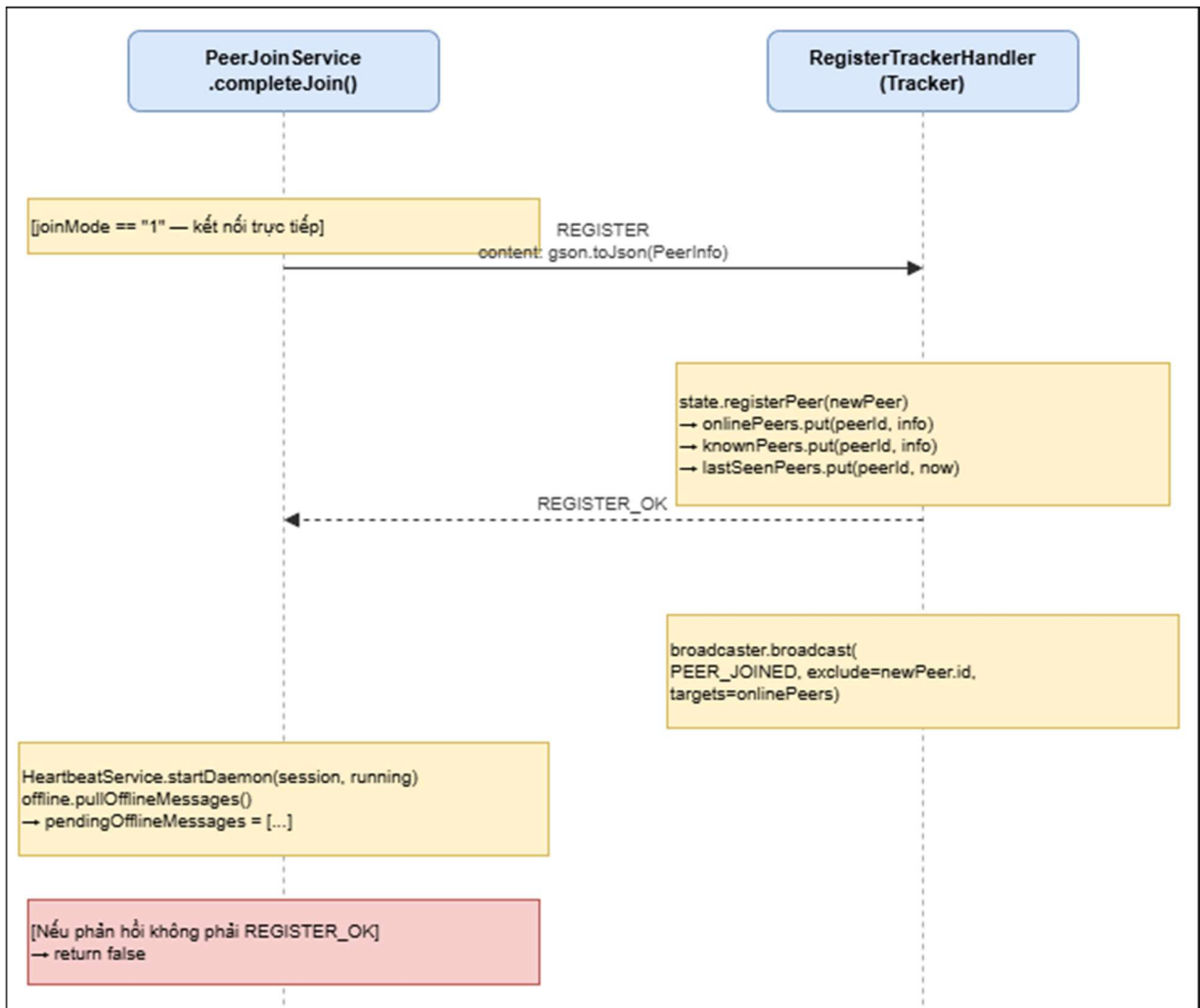
Cơ chế resume: Khi nhận PEER_JOINED của peer đang có transfer chờ, PeerChangedInboundHandler tự động gọi triggerResume() — sender hỏi FILE_RESUME_REQ để biết chunk tiếp theo cần gửi

CHƯƠNG 3: CƠ CHẾ PEER DISCOVERY

3.1. Đăng ký trực tiếp với Tracker

Khi một peer mới khởi động, bước đầu tiên là đăng ký với Tracker để thông báo sự hiện diện trong mạng. Luồng xử lý:

- Tạo đối tượng PeerInfo chứa ID, địa chỉ IP quảng bá (lấy từ application.properties), và port lắng nghe.
- Tuần tự hóa PeerInfo thành JSON và đóng vào message REGISTER.
- Gửi TCP đến địa chỉ Tracker (tracker.host:tracker.port).
- Đọc phản hồi: nếu REGISTER_OK → khởi động HeartbeatService, gửi PULL_OFFLINE, cache danh sách peer; nếu REGISTER_FAIL → báo lỗi.

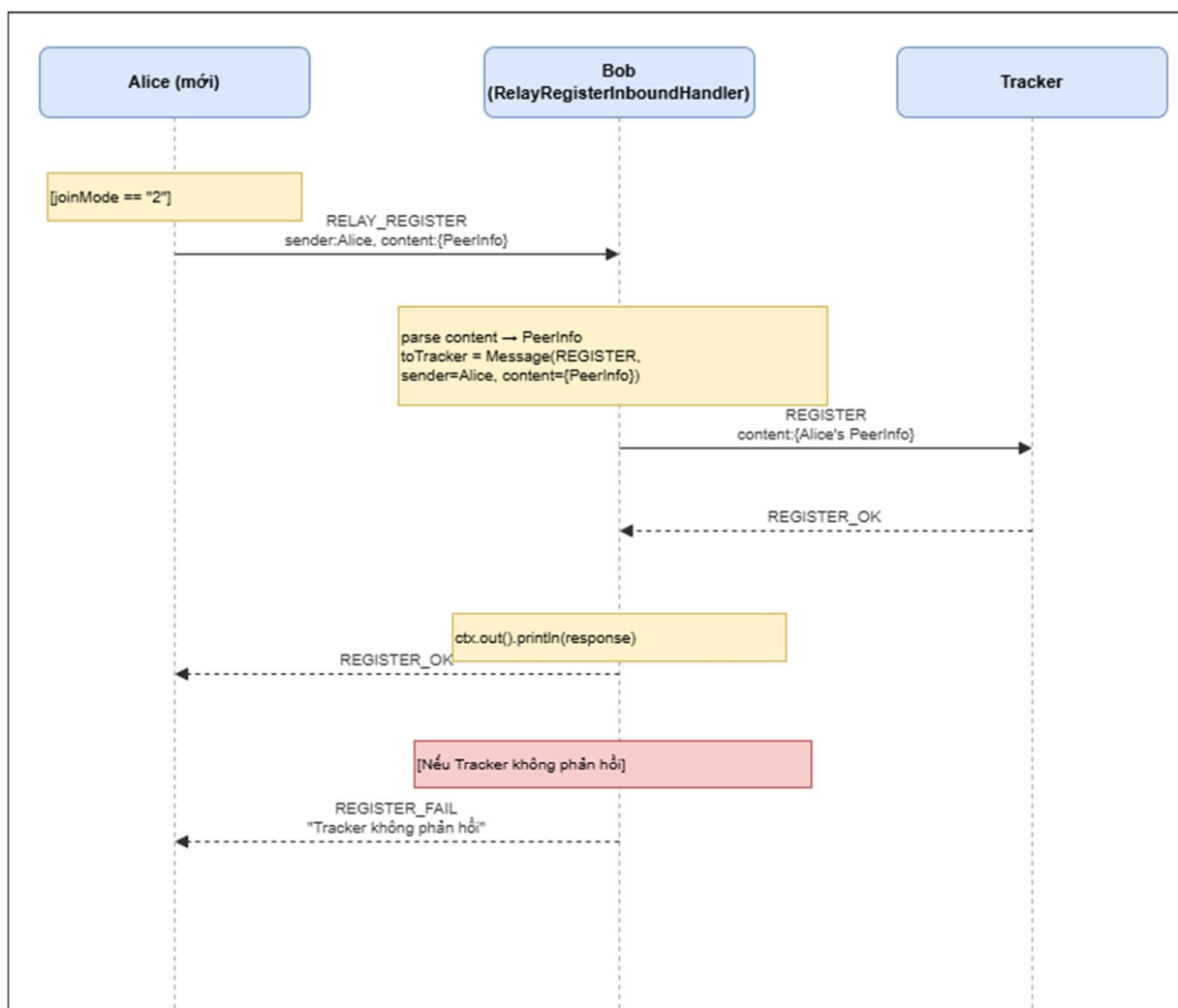


Hình 6. Luồng đăng ký trực tiếp với Tracker

3.2. Đăng ký qua peer trung gian

Trong trường hợp peer mới không thể kết nối trực tiếp đến Tracker do tường lửa hoặc cấu hình mạng, peer có thể nhờ một peer đã online làm cầu nối (relay register):

- Peer mới (Alice) gửi message RELAY_REGISTER đến Bob (peer đã online), kèm theo nội dung PeerInfo của mình.
- Bob nhận được qua RelayRegisterInboundHandler, tạo lại message REGISTER gốc và gửi đến Tracker.
- Bob nhận phản hồi từ Tracker và chuyển tiếp nguyên vẹn về cho Alice



Hình 7 Luồng đăng ký qua peer trung gian

3.3. Truy vấn danh sách peer online

Bất kỳ peer nào cũng có thể truy vấn danh sách peer đang online thông qua PeerDirectoryService:

- Gửi message GET_PEERS đến Tracker.

- Tracker phản hồi bằng message PEER_LIST với trường content chứa JSON array tất cả PeerInfo đang trong onlinePeers.
- Client giải tuần tự hóa và sử dụng để định tuyến tin nhắn hoặc hiển thị lên giao diện.

```
// Ví dụ phản hồi PEER_LIST
{
  "type": "PEER_LIST",
  "content": "[
    {\"peerId\": \"Alice\", \"ipAddress\": \"localhost\", \"port\": 5001},
    {\"peerId\": \"Bob\", \"ipAddress\": \"localhost\", \"port\": 5002}
  ]"
}
```

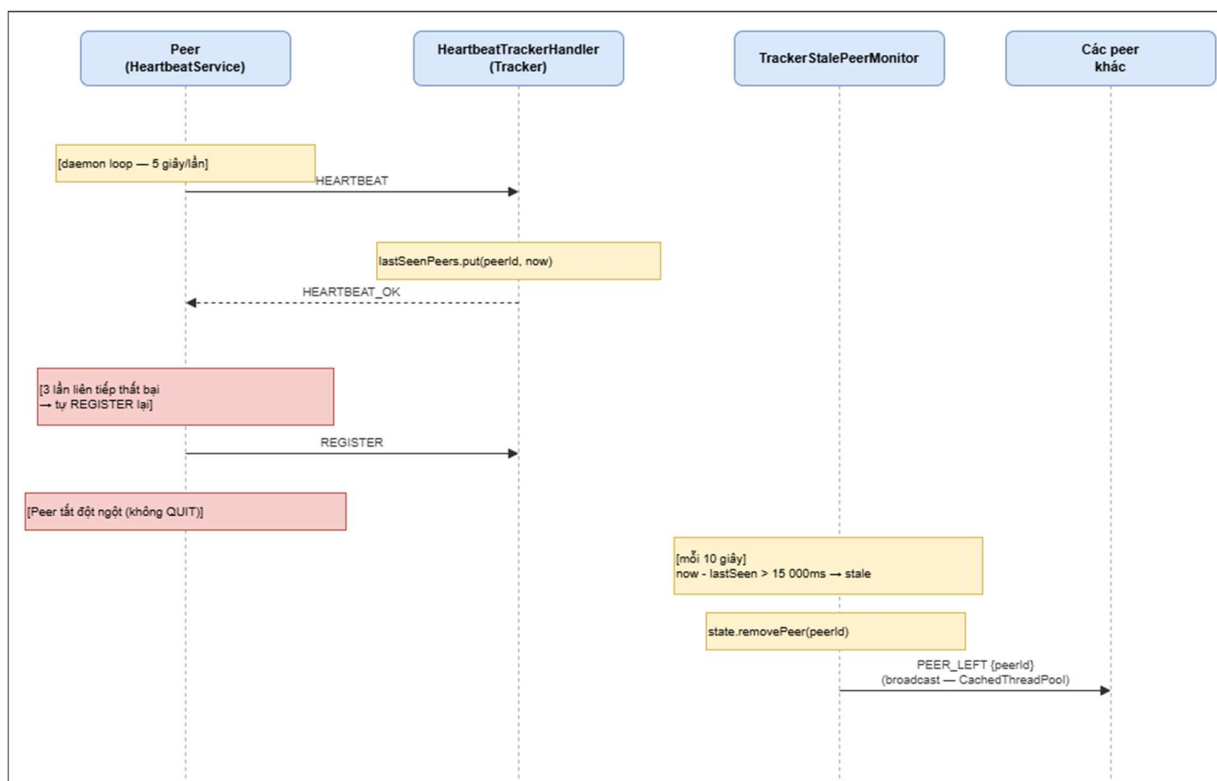
PeerDirectoryService cache kết quả 5 giây (TTL) và áp dụng stale-on-error: nếu Tracker không phản hồi, trả về danh sách cũ thay vì báo lỗi, đảm bảo trải nghiệm người dùng mượt mà.

3.4. Cơ chế Heartbeat và phát hiện peer mất kết nối

Vì TCP không tự động phát hiện peer im lặng, hệ thống triển khai cơ chế heartbeat chủ động hai lớp.

Bảng 8. Thông số cơ chế Heartbeat

Thành phần	Tham số	Mô tả
HeartbeatService	Interval: 5 giây Ngưỡng thử lại: 3 lần thất bại liên tiếp	Daemon thread gửi HEARTBEAT định kỳ; nếu Tracker không phản hồi 3 lần liên tiếp → tự động đăng ký lại (re-registration) mà không cần can thiệp của người dùng.
TrackerStalePeerMonitor	Kiểm tra: 10 giây Ngưỡng stale: 15 giây	Daemon thread của Tracker, quét toàn bộ lastSeenPeers và xóa peer nếu (now - lastSeen) > 15 giây, sau đó broadcast PEER_LEFT.



Hình 8. Cơ chế Heartbeat và phát hiện peer mất kết nối

Trong trường hợp xấu nhất (peer tắt ngay sau lần heartbeat cuối), Tracker cần tối đa $15 + 10 = 25$ giây để phát hiện và thông báo. Trong trường hợp trung bình (tắt ở giữa chu kỳ heartbeat 5 giây), thời gian phát hiện là $\sim 17,5$ giây.

3.5. Cập nhật danh sách peer real-time

Hệ thống triển khai cơ chế push event-driven: Tracker chủ động thông báo mọi thay đổi đến tất cả peer đang online, thay vì để peer phải polling định kỳ.

3.5.1. Thiết kế tổng thể

Cơ chế push hoạt động theo mô hình fire-and-forget: Tracker mở một kết nối TCP ngắn đến PeerServer của từng peer, gửi message PEER_JOINED hoặc PEER_LEFT, không đợi phản hồi. Connect timeout 2 giây tránh chặn thread.

3.5.2. TrackerPushBroadcaster

Lớp TrackerPushBroadcaster chịu trách nhiệm gửi thông báo đến các peer. Mỗi lần gọi broadcast(), nó tạo một task riêng cho mỗi peer đích trong CachedThreadPool để các push chạy song song, không bị chặn bởi peer chậm.

```
// Gửi tới tất cả peer (trừ excludePeerId nếu có)
```

```

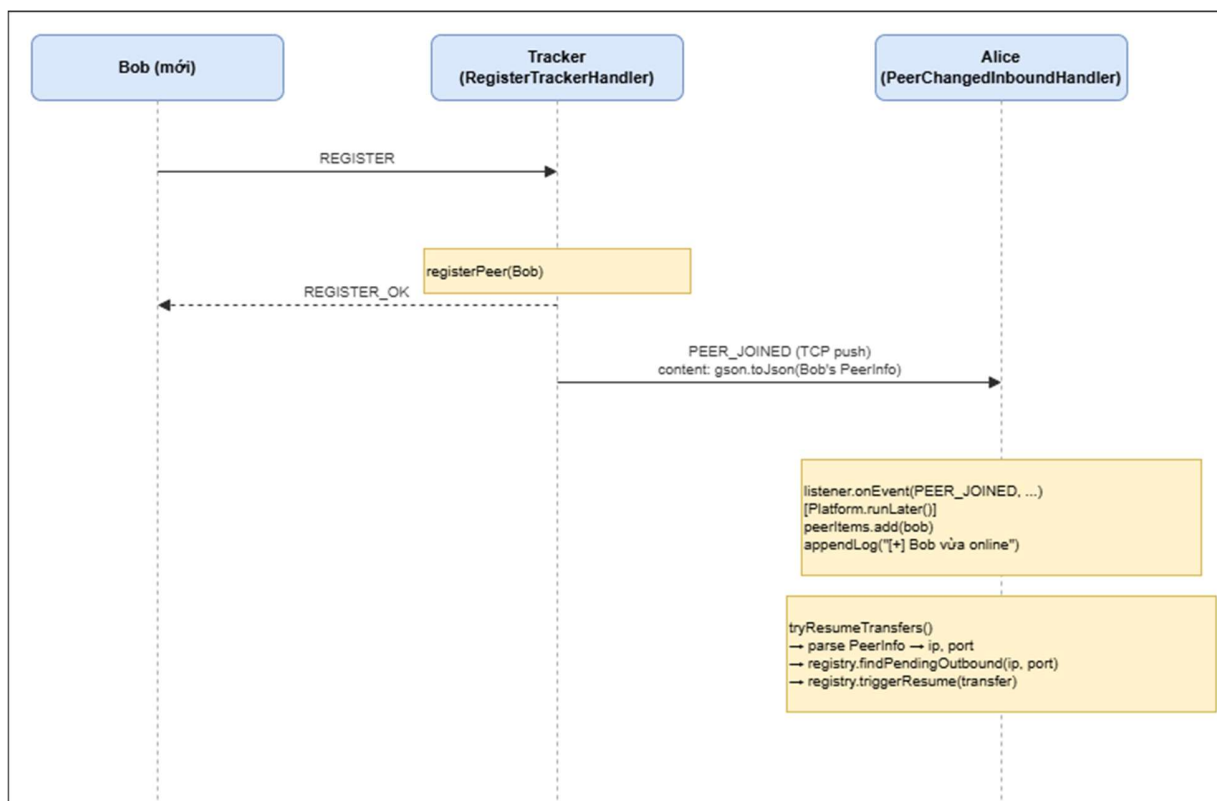
public void broadcast(Message msg, String excludePeerId, Collection<PeerInfo>
targets) {
    String json = gson.toJson(msg);
    for (PeerInfo peer : targets) {
        if (excludePeerId != null && excludePeerId.equals(peer.getPeerId()))
continue;
        pool.submit(() -> push(peer.getIpAddress(), peer.getPort(), json));
    }
}

private void push(String ip, int port, String json) {
    try (Socket socket = new Socket()) {
        socket.connect(new InetSocketAddress(ip, port), CONNECT_TIMEOUT_MS);
        new PrintWriter(socket.getOutputStream(), true).println(json);
    } catch (Exception ignored) {} // peer không reach được – bỏ qua silently
}

```

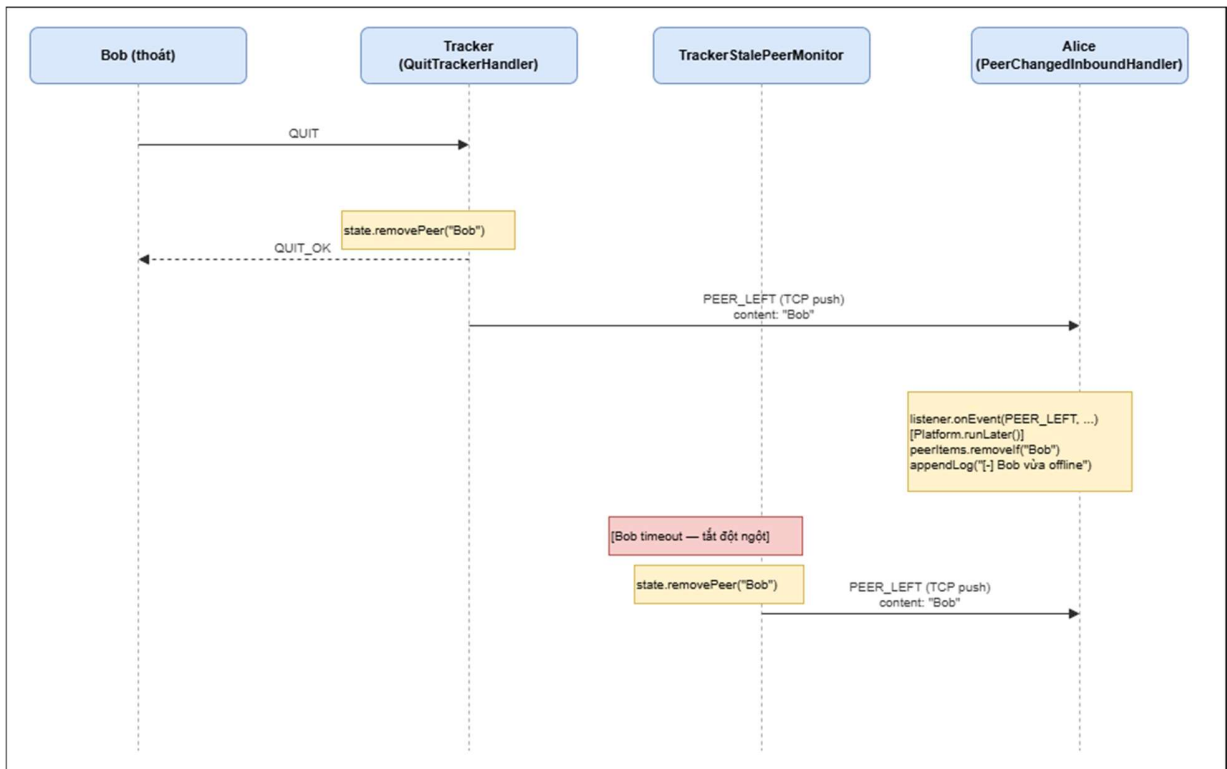
Connect timeout 2 giây đảm bảo thread không bị treo vô hạn khi peer đích đã tắt nhưng chưa bị StalePeerMonitor phát hiện.

3.5.3. Luồng sự kiện khi peer mới join



Hình 9 Luồng push **PEER_JOINED** khi peer mới tham gia mạng

3.5.4. Luồng sự kiện khi peer rời mạng



Hình 10. Luồng push PEER_LEFT khi peer rời mạng

3.5.5. Xử lý trên phía Peer nhận

Handler PeerChangedInboundHandler được đăng ký trong PeerMessageDispatcher cùng với các handler khác. Khi nhận message type PEER_JOINED hoặc PEER_LEFT:

- Không gửi ACK về (Tracker không đợi phản hồi).
- Chuyển event lên IncomingMessageListener.
- ChatController xử lý trên JavaFX Application Thread qua Platform.runLater(): với PEER_JOINED — parse JSON thành PeerInfo và thêm vào ObservableList; với PEER_LEFT — xóa peer khỏi danh sách theo peerId.

Tính chịu lỗi: Nếu push thất bại (peer không thể reach được), Tracker bỏ qua silently — danh sách peer trong UI có thể bị lệch tạm thời nhưng sẽ đồng bộ lại tại lần GET_PEERS tiếp theo (cache TTL 5 giây).

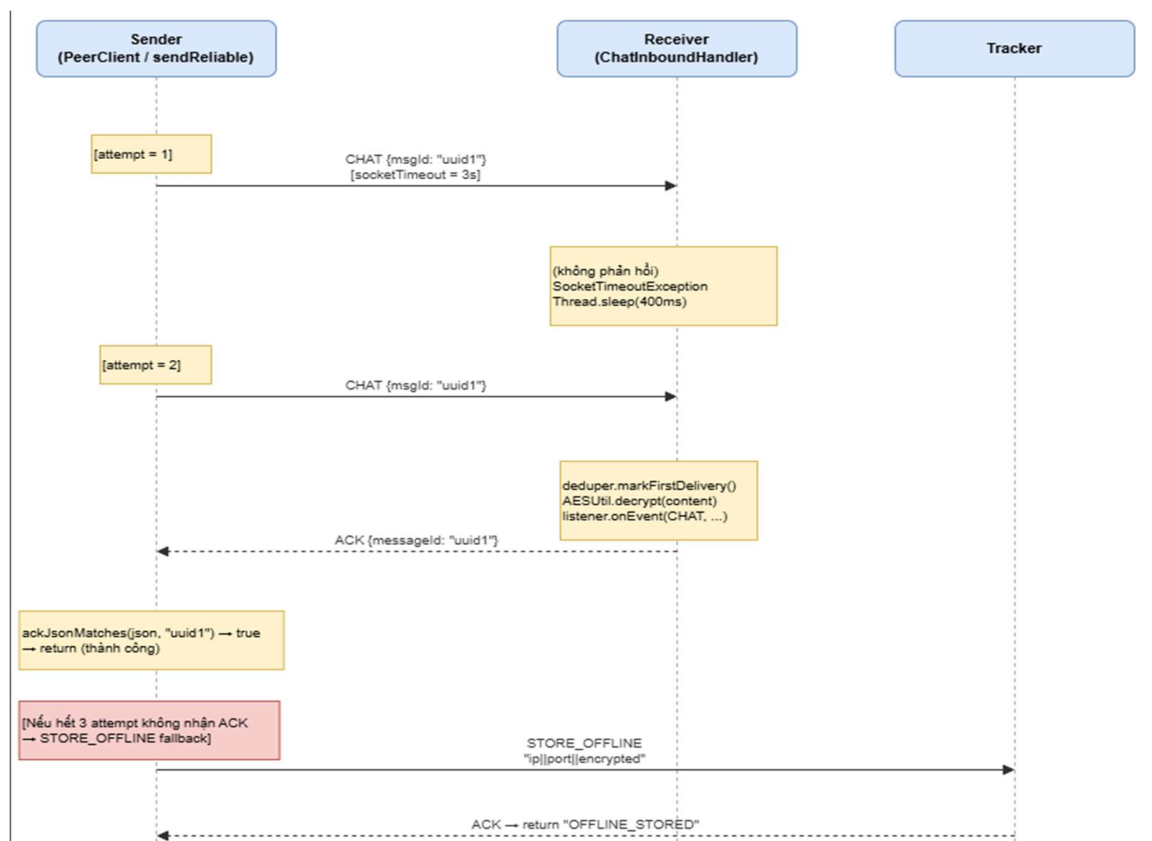
CHƯƠNG 4: XỬ LÝ LỖI HỆ THỐNG

4.1. Truyền tin đáng tin cậy

Tầng TCP đảm bảo dữ liệu đến đích theo thứ tự và không bị mất mát khi kết nối ổn định. Tuy nhiên TCP không đảm bảo khi peer đích crash hoặc không xử lý kịp. Hệ thống triển khai thêm lớp reliable delivery ở tầng ứng dụng với cơ chế ACK và retry.

Bảng 9 Thông số cơ chế Reliable Delivery

Tham số	Giá trị
Socket timeout	3 giây / lần thử
Số lần retry tối đa	3 lần
Delay giữa các retry	400ms
Cơ chế xác nhận	ACK với messageId trùng khớp
Timeout đọc phía server	10 giây (peer.server.read.timeout.ms)
Fallback khi hết retry	DirectChatService chuyển sang STORE_OFFLINE tại Tracker



Hình 11 Cơ chế retry và xác nhận ACK

4.2. Lưu tin khi peer offline

Khi gửi tin thất bại và hết retry, hệ thống không đơn giản báo lỗi mà cố gắng lưu tin vào Tracker để giao cho người nhận khi họ online trở lại:

- `DirectChatService.send()` gọi `sendReliable(ip, port, chatMsg)` — thất bại (trả về null sau 3 lần retry).
- Gọi trực tiếp `offline.tryStore(targetIp, targetPort, encryptedContent, msgId)` — không cần tra `peerId`.
- `tryStore()` đóng gói payload "ip||port||encrypted" (3 phần) và gửi `STORE_OFFLINE` tới Tracker.
- Tracker (`StoreOfflineTrackerHandler`) tra cứu `peerId` từ địa chỉ IP:Port (`findKnownPeerByAddress`), lưu vào `offlineMessages[peerId]`.

Khi người nhận đăng nhập, bước đầu tiên sau `REGISTER` là gửi `PULL_OFFLINE` → nhận toàn bộ tin đang chờ, Tracker xóa hàng đợi.

Bảo mật: Nội dung được mã hóa AES trước khi gửi lên Tracker, vì vậy Tracker chỉ lưu ciphertext, không đọc được nội dung tin nhắn.

Để ngăn một peer độc hại lấp đầy bộ nhớ Tracker, mỗi peer nhận có hàng đợi offline tối đa 50 tin nhắn. Nếu vượt ngưỡng, `STORE_OFFLINE` trả về `NACK`

4.3. Chống tin trùng

Do cơ chế retry, cùng một message có thể đến tay người nhận nhiều lần. Lớp `DeliveryIdempotencyTracker` giải quyết vấn đề này:

- Mỗi message được định danh bởi khóa tổng hợp: `type|senderId|messageId`.
- Khóa được lưu vào một LRU map (insertion-order `LinkedHashMap` bọc trong `Collections.synchronizedMap`) khi xử lý lần đầu.
- Các lần tiếp theo cùng khóa → bị bỏ qua, ACK vẫn được gửi lại để peer gửi không bị treo.
- Khi map vượt 4096 mục → evict entry cũ nhất (thay vì xóa toàn bộ), đảm bảo lịch sử gần nhất luôn được giữ lại.

```
String key = msg.getType() + "|" + msg.getSenderId() + "|" + msg.getMessageId();
synchronized (seen) {
    if (seen.containsKey(key)) return false;

    seen.put(key, Boolean.TRUE); // LinkedHashMap.removeEldestEntry() tự evict
    nếu size > 4096

    return true;.
}
```

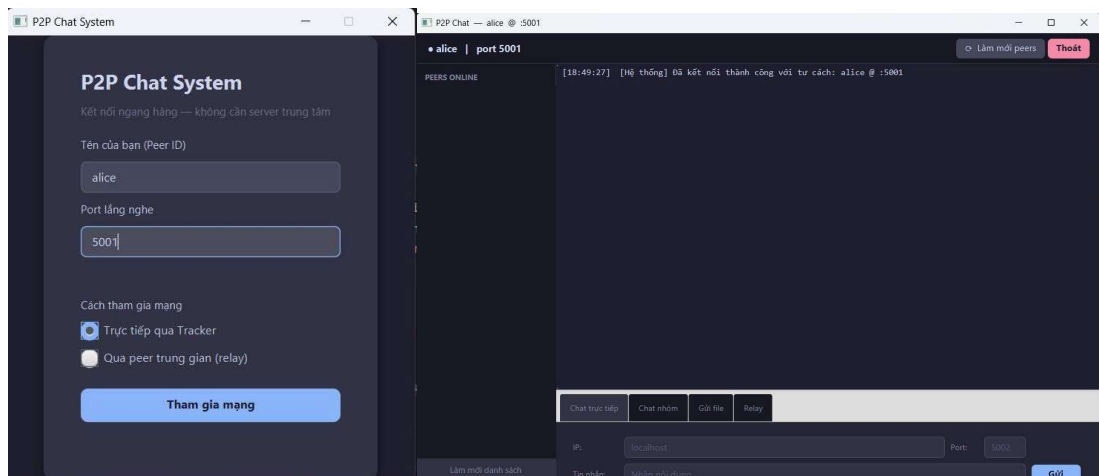

CHƯƠNG 5: THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1. Kịch bản thử nghiệm

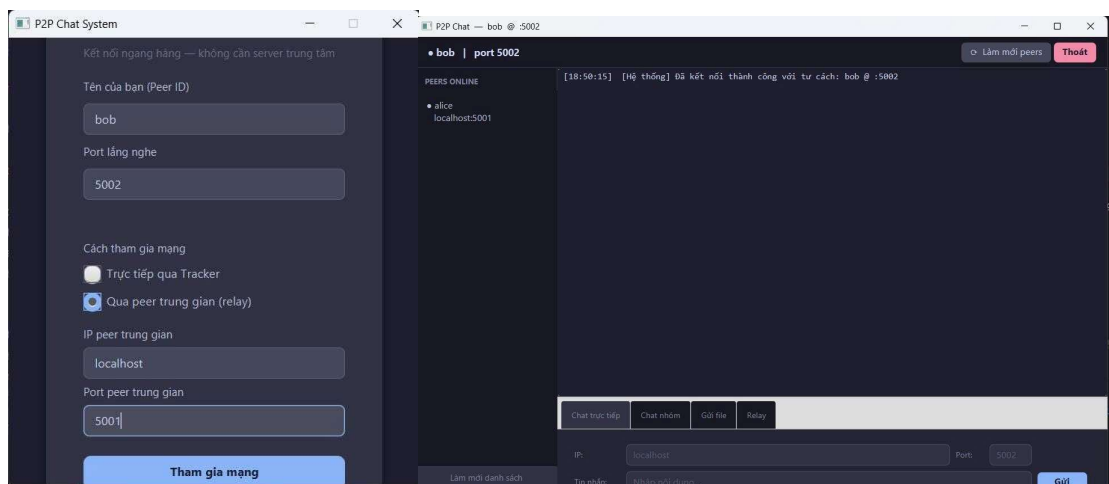
5.5.1: Thử nghiệm chức năng cơ bản

Bảng 10. Kịch bản thử nghiệm chức năng cơ bản

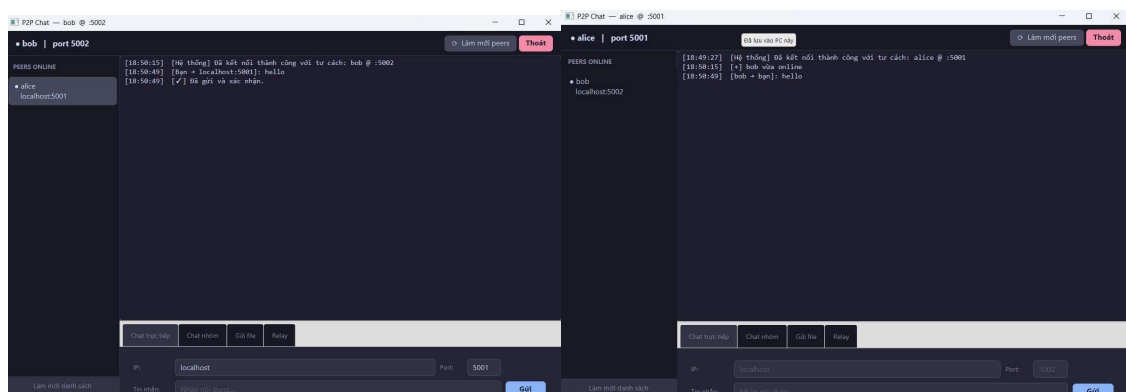
STT	Kịch bản	Điều kiện	Kết quả kỳ vọng
TC-01	Đăng ký trực tiếp với Tracker	Tracker đang chạy tại localhost:9000	Peer xuất hiện trong PEER_LIST; HeartbeatService khởi động
TC-02	Đăng ký qua peer relay	Tracker đang chạy, ít nhất 1 peer đã online	Peer mới xuất hiện trong PEER_LIST
TC-03	Chat một-một	2 peer đang online	Tin nhắn được giải mã đúng; ACK trả về trong < 3 giây
TC-04	Chat nhóm (GROUP_CHAT)	3+ peer đang online	Tất cả peer nhận tin cùng nội dung; CompletableFuture hoàn thành song song
TC-05	Gửi file nhỏ (< 512KB)	2 peer đang online	File xuất hiện trong downloads_<port>/ của receiver, nội dung trùng khớp
TC-06	Gửi file lớn (10MB, nhiều chunk)	2 peer đang online	File được lắp ráp đúng từ 20 chunk; hash MD5 trùng khớp
TC-07	Peer QUIT chủ động	Peer gửi QUIT trước khi tắt	PEER_LEFT broadcast; peer biến mất khỏi PEER_LIST của các peer khác
TC-08	Xem danh sách peer online	Nhiều peer đang online	PEER_LIST hiển thị đúng các peer; cache 5 giây hoạt động



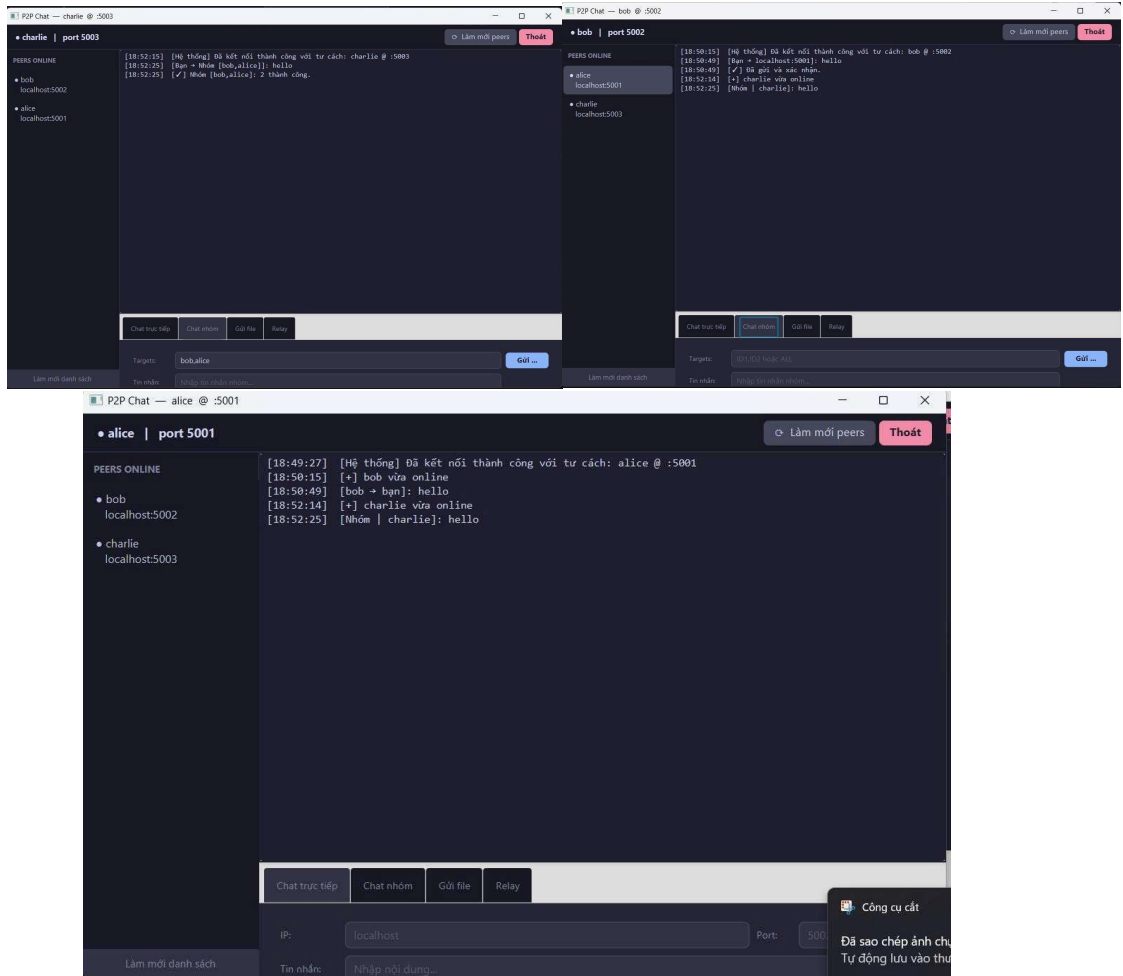
Hình 12. TC-01. Đăng ký trực tiếp với Tracker



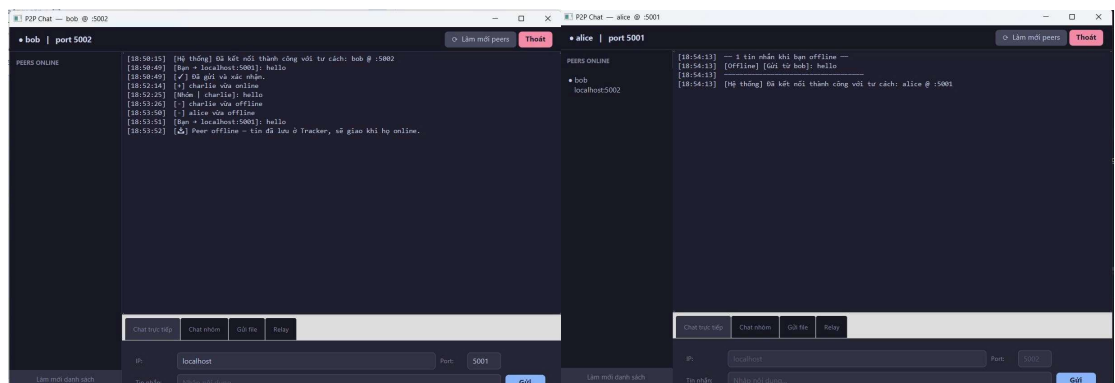
Hình 13. TC-02. Đăng ký qua peer relay



Hình 14. TC-03. Chat một-một



Hình 15. TC-04. Chat Nhóm



Hình 16. TC-09. Gửi tin khi peer đi về offline

5.5.1: Thử nghiệm chức năng cơ bản

Bảng 11. Kịch bản thử nghiệm xử lý lỗi

STT	Kịch bản	Điều kiện	Kết quả kỳ vọng
TC-09	Gửi tin khi peer đích offline	Bob đã tắt, Alice gửi tin	Sau 3 lần retry thất bại → tin được lưu vào Tracker (STORE_OFFLINE thành công)
TC-10	Bob online lại nhận tin offline	Bob đăng nhập sau TC-09	PULL_OFFLINE trả về tin từ Alice; Bob hiển thị đúng nội dung đã giải mã
TC-11	Tin nhắn trùng lặp (retry)	Gửi lại cùng messageId 3 lần	Chỉ hiển thị 1 lần; DeliveryIdempotencyTracker lọc bản trùng
TC-12	Peer tắt đột ngột (không QUIT)	Kill tiến trình peer	Sau 15–25 giây, Tracker xóa peer; PEER_LEFT broadcast đến các peer còn lại
TC-13	Resume truyền file	Ngắt kết nối Alice → Bob giữa chừng; Bob khởi động lại	PeerChangedInboundHandler kích hoạt resume; gửi từ chunk bị thiếu
TC-14	Tracker restart — peer tự tái kết nối	Tắt rồi bật lại Tracker	HeartbeatService phát hiện 3 lần thất bại → tự REGISTER lại không cần người dùng
TC-15	File vượt giới hạn 50MB	Gửi file 60MB	FileSendService từ chối ngay: "Kích thước file vượt quá giới hạn 50MB"
TC-16	Hàng đợi offline đầy	Gửi 51+ tin khi Bob offline	Từ tin thứ 51, Tracker trả về NACK; Alice nhận thông báo lưu thất bại
TC-17	Chat nhóm khi 1 thành viên offline	Alice gửi GROUP_CHAT tới Bob (online) và Carol (offline)	Bob nhận tin trực tiếp; Carol nhận qua PULL_OFFLINE khi đăng nhập lại; GroupChatService báo cáo: "Hoàn tất! 1 thành công, 1 lưu offline, 0 thất bại"
TC-18	Chat nhóm khi tất cả thành viên offline	Alice gửi GROUP_CHAT tới Bob và Carol, cả hai đều offline	GroupChatService thử sendReliable thất bại cho cả hai → STORE_OFFLINE thành công; báo cáo: "Hoàn tất! 0 thành công, 2 lưu offline, 0 thất bại"
TC-19	Chat nhóm – thành viên không tồn tại	Alice gửi GROUP_CHAT tới Bob (online) và	Bob nhận tin thành công; "unknown_peer" không tra được địa chỉ → GroupChatService đếm vào cột

		"unknown_peer" (không có)	thất bại; báo cáo: "Hoàn tất! 1 thành công, 0 lưu offline, 1 thất bại"
--	--	------------------------------	--

5.2. Mô phỏng churn mạng

Lớp ChurnSimulator tự động hóa việc mô phỏng nhiều peer tham gia và rời mạng liên tục, giúp kiểm tra tính ổn định của TrackerState và cơ chế broadcast dưới tải động.

Bảng 12. Thông số ChurnSimulator

Tham số (args)	Mặc định	Mô tả
numPeers (args[0])	5	Số peer ảo được tạo đồng thời
durationSec (args[1])	60	Tổng thời gian chạy mô phỏng (giây)
basePort (args[2])	6100	Port đầu tiên; mỗi peer ảo dùng basePort + index
Online duration / peer	3–15 giây (random)	Thời gian mỗi peer ảo ở lại mạng trước khi QUIT
Offline duration / peer	0.8–4 giây (random)	Thời gian nghỉ trước khi peer ảo REGISTER lại

Cách chạy:

```
mvn exec:java -Dexec.mainClass=org.khoicg.chat.sim.ChurnSimulator -Dexec.args="10 120"
# 10 peer ảo, chạy trong 120 giây, basePort mặc định 6100
```

Kết quả quan sát: Tracker xử lý hàng trăm lần REGISTER/QUIT mỗi phút mà không bị crash, danh sách PEER_LIST luôn phản ánh đúng trạng thái thực, không có deadlock hay race condition nhờ ConcurrentHashMap và synchronized block trong DeliveryIdempotencyTracker.

5.3. Đánh giá kết quả

Đồ án đã thiết kế và triển khai thành công một hệ thống P2P Chat đáp ứng đầy đủ các yêu cầu của đề bài, bao gồm:

Mô hình Hybrid P2P với Tracker hỗ trợ peer discovery và store-and-forward.

Giao tiếp TCP/JSON với giao thức thông điệp rõ ràng, đầy đủ các loại message cho các tình huống: chat, nhóm, file, relay, và offline.

Truyền tin đáng tin cậy với cơ chế ACK xác nhận, retry tự động, idempotency chống tin trùng, read timeout và giới hạn kích thước message.

Mã hóa AES-GCM bảo vệ nội dung, kể cả khi lưu tạm trên Tracker.

Truyền file theo chunk 512KB với resume tự động khi kết nối bị gián đoạn; kích thước tối đa 50MB; bảo vệ path traversal.

Ổn định dưới tải: thread pool cố định cho cả Tracker (10 threads) và Peer (20 threads); quota hàng đợi offline 50 tin/peer; tự dọn dẹp transfer hết hạn sau 24 giờ.

Khả năng mở rộng: broadcast nhóm thực hiện song song bằng CompletableFuture — thời gian gửi không tăng tuyến tính theo số peer; danh sách peer được cache 5 giây với stale-on-error.

Tự động tái kết nối khi Tracker restart: HeartbeatService phát hiện 3 lần thất bại liên tiếp và gửi lại REGISTER mà không cần can thiệp của người dùng.

Giao diện GUI hiện đại (JavaFX 21, dark theme) song song với CLI truyền thống; danh sách peer online tự động cập nhật real-time nhờ cơ chế push event-driven.

Công cụ ChurnSimulator phục vụ kiểm thử tự động dưới tải động.

Tracker Server có thể triển khai độc lập dưới dạng fat JAR (java -jar tracker.jar) mà không cần Maven hay thư viện ngoài.

Hướng cải tiến trong tương lai:

Triển khai xác thực peer bằng chứng chỉ số (PKI) để ngăn chặn giả mạo ID.

Thêm cơ chế distributed tracker (DHT – Kademlia) để loại bỏ điểm hỏng đơn (SPOF).

Hỗ trợ giao thức STUN/TURN để kết nối qua NAT trong môi trường Internet thực.

Trao đổi khóa AES bằng Diffie-Hellman để đạt E2EE thực sự thay vì shared key hardcoded.

Nâng cấp giao thức lên persistent connection (WebSocket/gRPC) để giảm overhead TCP handshake mỗi message.

Hướng cải tiến khi số lượng người dùng tăng (Scalability):

Horizontal scaling Tracker: Khi số peer vượt vài nghìn, một Tracker đơn sẽ trở thành bottleneck. Giải pháp là triển khai nhiều Tracker instance phía sau load balancer

(Nginx / HAProxy), chia sẻ trạng thái qua Redis hoặc cơ sở dữ liệu phân tán (Cassandra). Mỗi Tracker instance chỉ cần xử lý một phân vùng peer.

Connection pooling và persistent TCP: Hiện tại mỗi message mở một TCP connection mới, gây overhead 3-way handshake. Với hàng nghìn user gửi tin đồng thời, điều này làm tăng latency và cạn kiệt port. Cần chuyển sang connection pool (mỗi cặp peer giữ một kết nối lâu dài) hoặc dùng gRPC với multiplexing trên HTTP/2.

Message queue cho offline storage: Thay vì lưu offlineMessages trực tiếp trong bộ nhớ Tracker (bị mất khi restart), sử dụng message queue bền vững như Apache Kafka hoặc RabbitMQ. Tin offline được persist trên disk, đảm bảo không mất dữ liệu khi Tracker crash, và cho phép nhiều Tracker instance cùng đọc/ghi hàng đợi.

Rate limiting và throttling: Khi nhiều user đồng thời gửi GROUP_CHAT "ALL", Tracker bị flood bởi hàng trăm STORE_OFFLINE request cùng lúc. Cần thêm rate limiter per-peer (ví dụ: token bucket) để giới hạn số message mỗi peer có thể gửi trong một giây, tránh Tracker bị quá tải.

Phân vùng peer directory theo region (Geographic Sharding): Khi peer phân bố toàn cầu, GET_PEERS trả về toàn bộ danh sách gây lãng phí băng thông. Cần chia peer theo region/cluster; mỗi peer ưu tiên kết nối với peer trong cùng region để giảm latency, chỉ kết nối cross-region khi cần thiết.

Nâng cấp thread pool động (Adaptive Thread Pool): Thread pool cố định (10 Tracker, 20 Peer) phù hợp với tải thấp nhưng không co giãn khi tải tăng đột biến. Cần dùng ForkJoinPool hoặc virtual thread (Project Loom – Java 21) để xử lý hàng nghìn kết nối đồng thời với chi phí bộ nhớ thấp hơn

TÀI LIỆU THAM KHẢO

Andrew S. Tanenbaum, Maarten Van Steen. Distributed Systems: Principles and Paradigms, 3rd Edition. Pearson, 2017.

Oracle Corporation. Java SE 21 Documentation — `java.net.Socket`.
<https://docs.oracle.com/>, 2023.

OpenJFX Project. JavaFX 21 API Documentation. <https://openjfx.io/javadoc/21/>, 2023.

Google. Gson User Guide. <https://github.com/google/gson>, 2024.

NIST. Advanced Encryption Standard (AES) — FIPS 197. 2001.

RFC 9293. Transmission Control Protocol (TCP). IETF, 2022.